

实验 2：参考手册 —— 工业级线程池理论基础

1. 为什么需要线程池？

在高性能网络服务器（如 Nginx, Redis 6.0+, Memcached）中，线程池是处理并发任务的核心手段。

1.1 传统“一请求一线程”模型的缺陷

- 资源开销**：创建和销毁线程涉及内核数据结构（PCB/栈空间）的分配与回收，耗费 CPU 时间。
- 上下文切换**：当大量线程竞争 CPU 资源时，频繁的上下文切换会导致吞吐量急剧下降。
- 系统负载控制**：如果没有限制线程数量，在高并发环境下会导致系统内存溢出（OOM）或调度器过载。

1.2 线程池的本质：池化技术（Pooling）

线程池维护一定数量的工作线程，让它们常驻内存并循环执行待办任务，从而将“线程的创建”转化为“任务的调度”。

2. 核心机制详解

2.1 任务队列 (Task Queue)

任务队列是线程池的“缓冲区”。

- 单消费者 vs 多消费者**：线程池通常是一个单生产者、多消费者（SPMC）模型。
- 数据结构**：一般使用链表或循环数组。

2.2 惊群效应 (Thundering Herd)

定义：当一个任务进入队列并触发条件变量信号（`pthread_cond_broadcast`）时，所有正在休眠的线程都被唤醒，但最终只有一个线程能成功抢到锁并处理任务，其余线程又重新回到休眠状态。

后果：无效的上下文切换和锁竞争，导致性能抖动。

对策：使用 `pthread_cond_signal` 仅唤醒一个线程，或者采用更先进的无锁队列（Lock-free Queue）。

2.3 虚假唤醒 (Spurious Wakeup)

现象：`pthread_cond_wait` 可能会在没有收到信号的情况下返回。

原因：这通常由 OS 内核的调度优化或中断处理引起，是 POSIX 标准允许的行为。

解决方法：永远在 `while` 循环中检查等待条件，而不是使用 `if`。

```
// 正确做法
while (condition_is_false) {
    // 释放
    pthread_cond_wait(&cond, &mutex);
}
```

3. 优雅退出 (Graceful Shutdown)

一个工业级系统绝不能直接调用 `exit()` 强杀进程。

1. **停止接收新任务**：设置 `shutdown` 标志。
2. **清理存量任务**：唤醒所有阻塞中的工作线程。
3. **汇合 (Join) 与销毁**：等待每个线程完成手头工作并正常退出，最后释放锁和条件变量。

4. 从原理到代码：标准补全流程

4.1 Worker 线程的执行顺序

在本实验给出的 `threadpool.c` 框架中，`worker` 线程应严格遵循如下流程：

1. 先获取互斥锁，准备检查共享队列状态。
2. 如果任务队列为空且线程池尚未关闭，则进入条件变量等待。
3. 线程被唤醒后，需要重新检查：
 - 当前是否仍然没有任务
 - 是否已经进入关闭状态
4. 如果线程池已关闭且队列也为空，则释放锁并退出线程。
5. 如果队列中存在任务，则从**队头**取出一个任务节点，并在必要时同步修正尾指针。
6. 释放锁。
7. 在锁外执行任务函数，并释放任务节点内存。

这个顺序不能随意调换。特别要注意：

- **先等待，再判退出**：否则线程可能在还有存量任务时提前退出。
- **取完任务立刻解锁**：业务函数通常可能耗时很长，不能带着队列锁执行。
- **头指针变空时同步清空尾指针**：否则尾插时会出现悬挂指针问题。

4.2 Submit 的标准步骤

`thread_pool_submit` 不只是“把任务丢进队列”这么简单，它还要负责处理失败分支与关闭状态：

1. 检查参数是否合法：例如线程池指针或任务函数指针是否为空。
2. 在堆上申请一个新的 `Task` 节点，并完成任务函数、参数、链表指针的初始化。
3. 进入临界区后再次检查 `shutdown` 标志。
4. 若线程池已关闭，应拒绝该任务，并回收刚刚申请的任务节点。
5. 若队列为空，应同时更新头尾指针；若队列非空，则执行尾插。
6. 插入成功后，使用条件变量唤醒等待中的工作线程。
7. 退出临界区并返回结果状态。

之所以这里用 `signal` 而不是 `broadcast`，是为了避免“惊群效应”。

4.3 Destroy 的标准顺序

`thread_pool_destroy` 的推荐流程如下：

1. 先检查线程池指针是否为空。
2. 进入临界区后，判断线程池是否已经进入关闭状态，避免重复销毁。
3. 设置 `shutdown` 标志，表示后续不再接收新任务。
4. 使用 `pthread_cond_broadcast` 唤醒所有可能仍阻塞在条件变量上的工作线程。
5. 退出临界区。
6. 逐个等待工作线程结束。
7. 所有线程退出后，再销毁互斥锁、条件变量，并释放线程数组和线程池本体。

这里“先解锁再 join”非常关键。若在持锁状态下等待线程退出，工作线程可能正需要这把锁才能走到退出路径，从而产生死锁。

4.4 当前实验框架中的返回值约定

- `thread_pool_submit`：成功返回 0，失败返回 -1。
- `thread_pool_worker`：在线程池关闭且任务清空后，通过 `pthread_exit(NULL)` 正常退出。

5. 常见实现细节与易错点

5.1 锁的粒度

临界区只负责保护共享数据结构：

- 任务队列头尾指针
- `shutdown` 标志
- 条件变量等待与唤醒过程

不要在持锁状态下执行 `task->function(task->argument)`。否则线程池会退化成“同一时刻只有一个线程能干活”的串行模型。

5.2 为什么 Submit 里要二次检查 shutdown

即使创建 `Task` 节点成功，也不代表这个任务一定可以提交。因为在线程刚申请完节点、尚未加锁时，别的线程可能已经调用了 `destroy`。因此必须在持锁后再次检查 `shutdown`，并在关闭状态下回收新节点。

5.3 为什么 Destroy 默认不手动清空任务链表

本实验给出的推荐实现默认语义是：

- 已经入队的任务由工作线程继续取走并执行完；
- 当 `shutdown == 1` 且队列为空时，工作线程才退出。

因此，在正常路径中，任务节点会被工作线程逐个 `free`，`destroy` 本身不需要再额外遍历并释放任务链表。

5.4 与当前实验文件的对应关系

学生在本实验中实际需要阅读和补全的是 `threadpool.c` 中给出的 `Task`、`ThreadPool` 和三个核心函数框架。

6. Pthreads 相关 API 复习

API	功能描述	核心考点
<code>pthread_mutex_lock</code>	获取互斥锁	阻塞调用，注意死锁风险
<code>pthread_mutex_unlock</code>	释放互斥锁	应与 <code>lock</code> 成对出现，避免死锁或长时间占锁
<code>pthread_cond_wait</code>	等待条件变量	释放锁 + 睡眠 + 被唤醒后重新获取锁（原子操作）
<code>pthread_cond_signal</code>	唤醒一个等待线程	效率高，避免惊喜效应
<code>pthread_cond_broadcast</code>	唤醒所有等待线程	常用于关闭线程池时的清理操作
<code>pthread_exit</code>	当前线程主动结束运行	常用于工作线程在满足退出条件后干净退出
<code>pthread_join</code>	等待指定线程结束并回收其资源	常用于主线程在 <code>destroy</code> 中等待所有工作线程退出
<code>pthread_mutex_destroy</code>	销毁互斥锁对象	应在线程全部结束且不再使用该锁后调用
<code>pthread_cond_destroy</code>	销毁条件变量对象	应在线程全部结束且不再使用该条件变量后调用

在线程池的关闭流程中，这两个 API 通常是配套出现的：

- 工作线程在检测到“线程池已关闭且队列为空”后，调用 `pthread_exit` 结束自己。
- 主线程在 `thread_pool_destroy` 中对每个工作线程调用 `pthread_join`，确保所有线程真正退出后再销毁锁、条件变量和线程池内存。

另外，线程池中与锁和条件变量相关的三个阶段分别是：

- 运行时同步**：使用 `pthread_mutex_lock` / `pthread_mutex_unlock` 保护共享队列。
- 等待与唤醒**：使用 `pthread_cond_wait` / `signal` / `broadcast` 协调线程休眠与唤醒。
- 最终清理**：在线程全部退出后，再调用 `pthread_mutex_destroy` 与 `pthread_cond_destroy` 释放同步原语本身。